Ingeniería de Software

Seminario de Arquitecturas de Software: Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Equipo: **Digimons**

Integrantes y contribución individual

Integrante	**Pregunta(s) trabajada(s)**
José Alejandro Albanés Febles	Pregunta 1. Análisis de *trade-offs* (N-Capas vs. Clean/Onion/Hexagonal); Pregunta 4. Inversión de dependencias en la práctica
Manuel Enrique Cardoso Martínez	Pregunta 2. Simetría del hexágono vs. jerarquía de las capas
Jorge Luis Herrera Cecilia	Pregunta 3. DDD y *Bounded Context* en un dominio de seguros
Leonardo Peláez Ascención	Pregunta 5. El *Kitty Problem* y el Principio Abierto/Cerrado
Karla Díaz Saura	Pregunta 6. N-Capas frente a requisitos de atributos de calidad
Todo el equipo	Introducción, conclusiones, revisión cruzada de las respuestas y bibliografía

Repositorio del proyecto: `https://github.com/JoseAlejandro0508/Smart-Routes`

22 de octubre de 2026

Índice

1. Introducción **2**

2. Pregunta 1. Análisis de *trade-offs*: N-Capas frente a Clean, Onion y Hexagonal **2**

2.1. Situación planteada 2

2.2. El error fundamental de acoplamiento 3

2.3. Cómo lo resuelve la Regla de Dependencia de Martin 3

2.4. Cómo lo resuelven los Puertos y Adaptadores de Cockburn 4

2.5. Impacto en la testabilidad 5

2.6. Impacto en el cambio de SQL Server a MongoDB 5

2.7. Síntesis de *trade-offs* 6

3. Pregunta 2. La simetría del hexágono frente a la jerarquía de las capas **6**

3.1. El problema conceptual que resuelve la simetría 6

3.2. Por qué el hexágono no tiene seis lados por una razón numérica 7

3.3. Del Puerto y Adaptador a las interfaces de la Arquitectura Onion 8

3.4. Lo que el dibujo de capas no transmite (y lo que sí aporta) 9

4. Pregunta 3. DDD y *Bounded Context* en un dominio de seguros **9**

4.1. El problema: un único modelo de "Póliza" para todos 9

4.2. División en contextos delimitados 10

4.3. Cómo evitar que el modelo de Póliza se contamine 11

4.4. Comunicación desacoplada entre contextos 11

4.5. Relación con una arquitectura orientada al dominio y el Agente de Servicio . . . 12

5. Pregunta 4. La inversión de dependencias en la práctica **14**

5.1. Situación actual 14

5.2. a) Por qué este diseño viola los principios de Hexagonal, Clean y Onion 14

5.3. b) Rediseño 15

5.4. c) Pruebas aisladas, mantenibilidad y cambio de proveedor 17

6. Pregunta 5. El *Kitty Problem* y el Principio Abierto/Cerrado **19**

6.1. Aclaración sobre la fuente 19

6.2. Descripción del problema 19

6.3. Por qué la división por verbos o funciones llevó al problema 20
6.4. Solución: componentes `Rides` y `Kittens` como *plugins* 20
6.5. Relación con el OCP y con DDD / Onion / Clean 22

7. Pregunta 6. N-Capas frente a requisitos de atributos de calidad 23
7.1. El punto de partida 23
7.2. Qué son las capas abiertas y el *fast-lane reader* 23
7.3. Efecto sobre el Aislamiento de Capas y la mantenibilidad 23
7.4. El *sinkhole anti-pattern* y la regla 80/20 24
7.5. Por qué las capas abiertas no resuelven la escalabilidad ni la agilidad 24
7.6. Por qué Hexagonal o Clean son más adecuadas, y con qué matices 25

8. Conclusiones 26

Referencias bibliográficas 27

Declaración de Uso de la Inteligencia Artificial 28

1. Introducción

La arquitectura de un sistema no es otra cosa que el conjunto de decisiones que, una vez tomadas, resultan más costosas de revertir. Por esa razón, elegir cómo se organizan las dependencias entre los distintos componentes condiciona la facilidad con la que el sistema podrá probarse, desplegarse y, sobre todo, evolucionar. Richards y Ford (2020, cap. 2) lo resumen en lo que llaman la *Primera Ley de la Arquitectura de Software*: todo en arquitectura es un compromiso (*trade-off*). No existe un estilo "correcto" en abstracto, sino estilos más o menos adecuados para un conjunto concreto de atributos de calidad.

En este seminario se estudian cinco enfoques que comparten la idea de organizar el software en capas, pero que difieren radicalmente en *hacia dónde apuntan las dependencias* y en *qué se coloca en el centro* del sistema:

- La **Arquitectura N-Capas tradicional**, organizada en capas horizontales (presentación, negocio, persistencia, base de datos) con dependencias de arriba hacia abajo (Richards, 2015, pp. 1–2).
- La **Arquitectura Hexagonal** o de **Puertos y Adaptadores**, que separa el interior de la aplicación de cualquier agente externo (Cockburn, 2005).
- La **Arquitectura Cebolla** (*Onion*), en la que el Modelo de Dominio ocupa el centro y todo el acoplamiento se dirige hacia él (Palermo, 2008a).
- La **Arquitectura Limpia** (*Clean Architecture*), que integra las anteriores bajo la *Regla de Dependencia* (Martin, 2017, p. 203).
- El **Diseño Guiado por el Dominio** (DDD), que aporta el vocabulario y los patrones para modelar aquello que las arquitecturas anteriores intentan proteger (Evans, 2003; De la Torre et al., 2010, pp. 41–43).

El documento se estructura en seis secciones, una por cada pregunta planteada en la orientación del seminario. En cada una se combinan los conceptos de la bibliografía con un caso práctico, ejemplos de código y diagramas elaborados por el equipo. Al final se presentan unas conclusiones generales que integran lo analizado.

2. Pregunta 1. Análisis de *trade-offs*: N-Capas frente a Clean, Onion y Hexagonal

2.1. Situación planteada

Un equipo eligió una arquitectura N-Capas para un sistema de gestión de inventario por ser simple y conocida. Un año después el sistema es un monolito difícil de mantener: los cambios en la base de datos se propagan a la lógica de negocio y a la interfaz de usuario, y las pruebas unitarias son lentas y frágiles.

Conviene reconocer primero que la decisión inicial no fue absurda. Richards y Ford (2020, cap. 10) señalan que el estilo en capas es una buena elección para aplicaciones pequeñas, con presupuesto ajustado o cuando todavía no se sabe qué estilo convendrá, precisamente por su simplicidad y bajo coste. El problema aparece cuando el sistema crece: en ese momento, según los mismos autores, la mantenibilidad, la agilidad, la testabilidad y la facilidad de despliegue se ven afectadas negativamente.

2.2. El error fundamental de acoplamiento

El error de fondo es que **la lógica de negocio depende, en tiempo de compilación, de la capa de acceso a datos**. En una arquitectura N-Capas clásica, cada capa depende de la que tiene debajo: la presentación conoce al negocio, el negocio conoce a la persistencia y la persistencia conoce a la base de datos. Palermo (2008a) identifica justamente este punto como "el mayor infractor": el acoplamiento de la interfaz y de la lógica de negocio con el acceso a datos. Y añade una observación que explica por qué los cambios en la base de datos llegan hasta la interfaz: *las dependencias transitivas siguen siendo dependencias*. Si la interfaz depende del negocio y el negocio depende del acceso a datos, la interfaz también depende, indirectamente, del acceso a datos.

En el sistema de inventario esto se manifiesta de forma concreta: una clase como `ServicioInventario` importa directamente un `ProductoDAO` que trabaja con `SqlConnection`, recibe filas o entidades generadas por el ORM y las devuelve tal cual hacia la presentación. Martin (2017, p. 278) califica esta práctica como un error arquitectónico, porque acopla los casos de uso, las reglas de negocio e incluso la interfaz a la estructura relacional de los datos.

Además, Richards y Ford (2020, cap. 10) explican que la arquitectura en capas está *particionada técnicamente*: el dominio "inventario" queda repartido entre todas las capas, de modo que un cambio en una regla de negocio obliga a tocar varias de ellas. A ello se suma lo que Cockburn (2005) describe como el problema de que la gente "no se toma en serio las líneas" del dibujo en capas: no hay ningún mecanismo que impida que la lógica de negocio se filtre a través de las fronteras.

La Figura 1 resume la diferencia en la dirección de las dependencias.

2.3. Cómo lo resuelve la Regla de Dependencia de Martin

Martin (2017, p. 203) formula la Regla de Dependencia así: *las dependencias de código fuente solo deben apuntar hacia adentro, hacia las políticas de más alto nivel*. Nada de un círculo interno puede conocer nada de un círculo externo; ni siquiera el nombre de una clase, función o variable declarada fuera. Esto incluye los formatos de datos: un círculo interno no debe recibir estructuras definidas por un *framework* externo (Martin, 2017, p. 204).

Aplicado al inventario, la reestructuración consiste en:

1. Colocar en el centro las **entidades** (`Producto`, `Almacen`, `MovimientoStock`) con sus reglas críticas, como "no puede haber stock negativo".

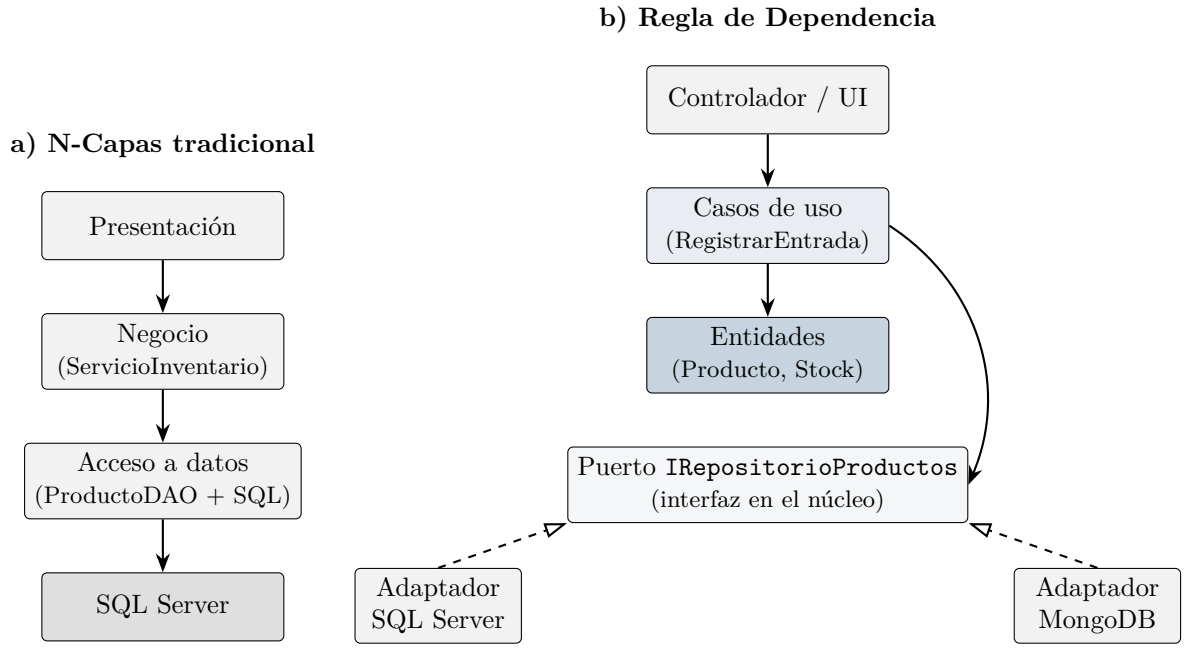


Figura 1: Dirección de las dependencias de código fuente. En (a) el negocio depende de la tecnología de datos; en (b) los adaptadores de infraestructura dependen de una abstracción definida en el núcleo. Elaboración propia a partir de Martin (2017, pp. 203–206) y Palermo (2008c).

2. Rodearlas de los **casos de uso** (`RegistrarEntrada`, `TransferirStock`), que orquestan las entidades.

3. Declarar en esa capa interna las **interfaces** que el caso de uso necesita, por ejemplo `IRepositorioProductos`.

4. Mover la implementación concreta (`RepositorioProductosSql`) a la capa de **adaptadores de interfaz**, que es la única que conoce SQL (Martin, 2017, p. 205).

El flujo de control sigue yendo del caso de uso a la base de datos, pero la dependencia de código fuente se *invierte*: el adaptador SQL depende de la interfaz del núcleo, y no al revés. Martin (2017, p. 206) explica que esta aparente contradicción se resuelve con el Principio de Inversión de Dependencias, utilizando polimorfismo dinámico para que las dependencias se opongan al flujo de control justo en las fronteras.

2.4. Cómo lo resuelven los Puertos y Adaptadores de Cockburn

Cockburn (2005) llega a la misma solución desde otra perspectiva. Plantea que tanto el problema del lado del usuario como el del lado de la base de datos tienen la misma causa: *el enredo entre la lógica de negocio y la interacción con entidades externas*. La regla es que el código del interior no debe filtrarse al exterior.

En términos del inventario:

- Un **puerto** es una conversación con propósito, expresada como una API. Por ejemplo, “obtener y guardar productos” o “registrar movimientos”.

- Un **adaptador** convierte esa API en las señales que necesita una tecnología concreta. Cockburn (2005) menciona explícitamente que para un mismo puerto de datos puede haber un adaptador SQL, uno de ficheros planos y, "lo más importante", un adaptador a una base de datos simulada en memoria.

Desde el punto de vista de la aplicación, si la base de datos cambia de SQL a otro tipo de almacenamiento, *la conversación a través de la API no debería cambiar* (Cockburn, 2005).

2.5. Impacto en la testabilidad

La lentitud y fragilidad de las pruebas actuales tiene una explicación directa: para probar una regla de negocio hay que levantar la base de datos, porque la clase de negocio crea o recibe objetos concretos de acceso a datos. Con la Regla de Dependencia, las reglas de negocio pueden probarse *sin la interfaz de usuario, la base de datos, el servidor web ni ningún otro elemento externo* (Martin, 2017, p. 202). Basta con inyectar un repositorio en memoria que implemente `IRepositorioProductos`.

Esto concuerda con el orden de desarrollo que propone Cockburn (2005): primero se conduce la aplicación con pruebas automáticas contra una base de datos simulada, después se añade la interfaz gráfica y, solo en las pruebas de integración, se conecta la base de datos real. La Guía de Arquitectura N-Capas de Microsoft también destaca esta ventaja: el desacoplamiento permite sustituir el módulo de acceso a datos reales por uno que "simplemente simula" el acceso, lo que habilita el uso de *stubs* y *mocks* (De la Torre et al., 2010, p. 65).

Martin (2017, pp. 251–252) añade un matiz importante sobre la fragilidad: las pruebas fuertemente acopladas a la estructura del sistema (por ejemplo, las que verifican reglas de negocio navegando por la interfaz) se rompen ante el más mínimo cambio. La solución es *diseñar para la testabilidad*, cuya primera regla es no depender de lo volátil.

2.6. Impacto en el cambio de SQL Server a MongoDB

Martin (2017, p. 202) menciona este escenario de forma literal como una de las características de la Arquitectura Limpia: *se puede sustituir Oracle o SQL Server por Mongo, BigTable, CouchDB u otra cosa*, porque las reglas de negocio no están ligadas a la base de datos. Con el diseño propuesto, el cambio se limita a:

1. Escribir un nuevo adaptador `RepositorioProductosMongo` que implemente la misma interfaz.
2. Cambiar el registro en el punto de composición (el contenedor de inyección de dependencias o el `Main`).

Ni las entidades ni los casos de uso se recompilan. Para Martin (2017, p. 278), la base de datos es un *detalle*, equivalente al pomo de la puerta respecto a la arquitectura de una casa.

Sin embargo, el equipo considera honesto señalar dos limitaciones. Primero, Martin (2017, p. 171) reconoce que estos reemplazos *pueden no ser triviales*; lo que garantiza la arquitectura es que el cambio sea *factible* y localizado, no gratuito. Segundo, pasar de un modelo relacional a uno documental puede revelar que la interfaz del repositorio estaba "contaminada" con supuestos relacionales (consultas con *joins*, transacciones entre varias tablas). Si el puerto se diseñó pensando en las necesidades del dominio y no en las tablas, el impacto será mínimo; si no, habrá que ajustarlo. Este es, de hecho, el motivo por el que Martin insiste en que los datos que cruzan las fronteras sean estructuras simples, convenientes para el círculo interno, y nunca filas de base de datos (Martin, 2017, p. 207).

2.7. Síntesis de *trade-offs*

Atributo	**N-Capas tradicional**	**Clean / Onion / Hexagonal**
Simplicidad inicial	Alta: estilo conocido y barato (Richards & Ford, 2020, cap. 10)	Menor: más interfaces, inyección de dependencias y mapeos entre modelos
Testabilidad del negocio	Depende de la infraestructura	Alta: núcleo probado de forma aislada (Martin, 2017, p. 202)
Cambio de base de datos	Afecta a negocio y presentación	Localizado en un adaptador
Curva de aprendizaje	Baja	Media: exige dominar DIP e inyección de dependencias
Adecuación	Aplicaciones pequeñas o centradas en datos	Aplicaciones de larga vida y lógica compleja (Palermo, 2008a)

Cuadro 1: Comparación de compromisos entre ambos enfoques. Elaboración propia.

En conclusión, el equipo de inventario no se equivocó al empezar con capas, sino al no reconocer el momento en que la complejidad del dominio exigía invertir las dependencias. Palermo (2008a) lo advierte desde el principio: su arquitectura no es apropiada para sitios pequeños, sino para aplicaciones empresariales de larga vida y comportamiento complejo, que es exactamente en lo que se convirtió el sistema de inventario.

3. Pregunta 2. La simetría del hexágono frente a la jerarquía de las capas

3.1. El problema conceptual que resuelve la simetría

Cuando se dibuja una arquitectura en capas apiladas, el dibujo transmite, sin decirlo, una idea de *jerarquía vertical*: la interfaz "arriba" y la base de datos "abajo". Esta imagen sugiere que la base de datos es el cimiento sobre el que se construye todo, y que la interfaz de usuario y la base de datos son cosas de naturaleza completamente distinta.

Cockburn (2005) identifica dos problemas en estos dibujos. El primero, y el peor, es que la gente no se toma en serio las líneas entre capas y deja que la lógica de la aplicación las atraviese.

El segundo es que muchas aplicaciones tienen más de dos puertos, de modo que la arquitectura no cabe en un dibujo unidimensional.

La aportación conceptual del hexágono es cambiar la pregunta. La asimetría que importa **no es entre la izquierda y la derecha, ni entre arriba y abajo, sino entre el interior y el exterior** de la aplicación (Cockburn, 2005). Desde el punto de vista de la aplicación, una persona usando la interfaz gráfica, un *script* de pruebas, otro sistema que llama por HTTP y una base de datos son, todos ellos, *agentes externos* con los que se mantiene una conversación a través de un puerto. La simetría del dibujo obliga a tratarlos de manera uniforme.

Este cambio de perspectiva tiene una consecuencia práctica muy fuerte: revela que el problema de "la lógica de negocio filtrándose en la interfaz" y el problema de "la lógica de negocio atada a la base de datos" *son el mismo problema*. Cockburn (2005) lo dice expresamente: no es obvio que ambos problemas estén relacionados, pero existe una simetría entre ellos que se hace visible en la naturaleza de la solución. Un dibujo en capas, al colocar la interfaz y la base de datos en extremos opuestos, oculta esa relación.

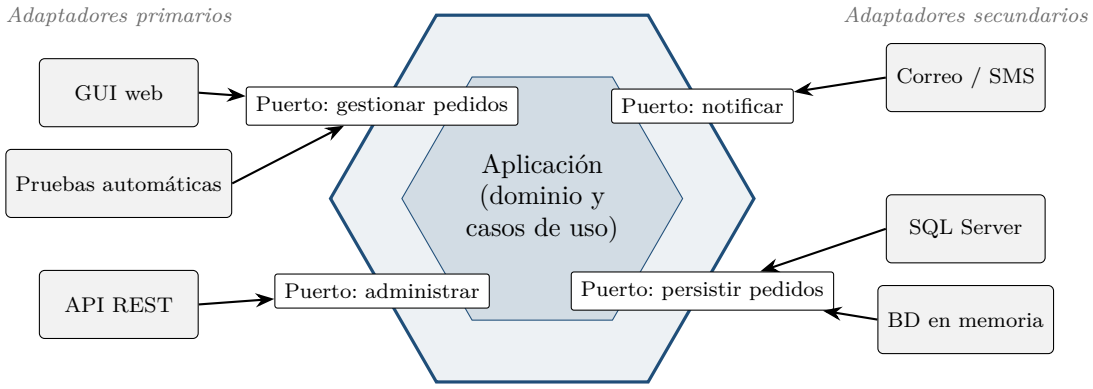


Figura 2: Arquitectura Hexagonal: varios adaptadores pueden conectarse a un mismo puerto. Las flechas indican dependencia de código: los adaptadores conocen el puerto, el núcleo no conoce los adaptadores. Elaboración propia a partir de Cockburn (2005).

3.2. Por qué el hexágono no tiene seis lados por una razón numérica

Cockburn (2005) es explícito: *el hexágono no es un hexágono porque el número seis sea importante*, sino para dar a quien dibuja espacio para insertar los puertos y adaptadores que necesite, sin la restricción de un dibujo en capas unidimensional. El término "arquitectura hexagonal" surge de ese efecto visual. El hexágono busca resaltar dos cosas: (a) la asimetría interior-exterior y la naturaleza similar de todos los puertos, alejándose de la imagen en capas "y de todo lo que evoca"; y (b) la existencia de un número definido de puertos diferentes, que en su experiencia suele ser dos, tres o cuatro.

En otras palabras, el hexágono es una herramienta de *comunicación*, no una prescripción estructural. Un círculo habría cumplido la función de mostrar "dentro y fuera", pero no ofrece lados donde ubicar puertos. Un rectángulo, en cambio, invita de nuevo a pensar en "arriba y abajo". La forma hexagonal elimina esas connotaciones y deja espacio suficiente. Por eso el nombre más preciso del patrón es *Puertos y Adaptadores*, que describe la función de cada parte

del dibujo (Cockburn, 2005).

Cabe destacar que el propio Cockburn reintroduce más adelante una distinción izquierda-derecha, pero con otro significado: los puertos *primarios* (o conductores), que impulsan a la aplicación, se dibujan a la izquierda; y los *secundarios* (o conducidos), a los que la aplicación da órdenes o hace consultas, a la derecha. Esta distinción proviene de los actores primarios y secundarios de los casos de uso (Cockburn, 2005). Lo importante es que esta asimetría *se deriva de quién inicia la conversación*, no de una jerarquía de capas, y se aplica como consecuencia de usar el patrón, no como sustituto.

3.3. Del Puerto y Adaptador a las interfaces de la Arquitectura Onion

Palermo (2008a) reconoce que la Arquitectura Hexagonal y la Onion comparten la misma premisa: *externalizar la infraestructura y escribir código adaptador para que la infraestructura no quede fuertemente acoplada*. La diferencia es de énfasis y de representación. Las cuatro reglas que Palermo (2008c) establece para su arquitectura permiten ver la correspondencia:

1. La aplicación se construye alrededor de un modelo de objetos independiente.
2. **Las capas internas definen interfaces; las capas externas las implementan.**
3. La dirección del acoplamiento es hacia el centro.
4. Todo el código del núcleo puede compilarse y ejecutarse por separado de la infraestructura.

La segunda regla es, precisamente, la traducción de "puerto y adaptador" al lenguaje de capas concéntricas:

- El **puerto** de Cockburn corresponde a una **interfaz definida en el núcleo** de la cebolla. Palermo (2008a) indica que la primera capa alrededor del Modelo de Dominio suele contener las *interfaces de repositorio*, que proporcionan el comportamiento para guardar y recuperar objetos.
- El **adaptador** de Cockburn corresponde a la **clase de infraestructura** situada en la capa externa que implementa esa interfaz. Palermo (2008a) explica que esa clase está acoplada a un método particular de acceso a datos, y por eso reside fuera del núcleo.

El ejemplo de CodeCampServer lo ilustra (Palermo, 2008b): `SpeakerController` (interfaz de usuario) depende solo de `IConferenceRepository` e `IUserSession`, interfaces definidas en el núcleo. Las clases `ConferenceRepository` y `UserSession` viven en capas externas e implementan esas interfaces. En tiempo de ejecución, un contenedor de Inversión de Control construye las clases adecuadas y las inyecta por el constructor. Es exactamente la mecánica de puertos y adaptadores: un adaptador secundario (el repositorio) que se conecta a un puerto (la interfaz), con el núcleo ignorante de qué hay al otro lado.

La Guía de Arquitectura N-Capas formaliza esta misma idea como una regla de diseño: definir los interfaces de los repositorios dentro de la Capa de Dominio, siguiendo el patrón *Separated*

Interface, y ubicar su implementación en la capa de Infraestructura de Persistencia (De la Torre et al., 2010, pp. 174–175).

3.4. Lo que el dibujo de capas no transmite (y lo que sí aporta)

El equipo considera que ambas metáforas tienen fortalezas complementarias:

- **El hexágono transmite mejor la equivalencia entre agentes externos.** En la cebolla, la interfaz de usuario, la infraestructura y las pruebas aparecen en el mismo anillo exterior (Palermo, 2008a), pero el dibujo no enfatiza que sean intercambiables respecto a un mismo puerto. En el hexágono, la imagen de varios adaptadores conectados al mismo puerto hace evidente que la GUI y un *script* de pruebas son, para la aplicación, lo mismo.
- **La cebolla transmite mejor la estructura interna del núcleo.** El hexágono trata el interior como una caja; no dice nada sobre cómo organizar el dominio. La cebolla, en cambio, diferencia el Modelo de Dominio en el centro, los servicios de dominio y los servicios de aplicación alrededor. Palermo (2008c) incluso muestra que, una vez aplanada, la cebolla se puede dibujar como capas apiladas, pero con el acceso a datos *arriba*, junto a la interfaz, y no abajo.

Este último punto es revelador: Palermo (2008c) dibuja la arquitectura en capas tradicional en forma de círculos concéntricos y concluye que, en ese caso, "se ve claramente que la aplicación está construida alrededor del acceso a datos". Es decir, **la forma del dibujo no es lo que define la arquitectura; lo que la define es la dirección de las dependencias**. Martin (2017, p. 247) llega a la misma conclusión al afirmar que la arquitectura de un sistema está definida por las fronteras que se trazan dentro de él y por las dependencias que cruzan esas fronteras.

4. Pregunta 3. DDD y *Bounded Context* en un dominio de seguros

4.1. El problema: un único modelo de "Póliza" para todos

La empresa aseguradora necesita un sistema de pólizas de automóviles que interactúa con siniestros, facturación de primas y una aplicación móvil para ajustadores. El enunciado muestra el núcleo del problema: la palabra *póliza* significa cosas distintas para cada área. Para un actuario es un contrato con cláusulas; para facturación, una fuente de ingresos; para un ajustador, un número de referencia.

La tentación clásica sería crear una única clase `Poliza` compartida por toda la organización, con decenas de atributos: cláusulas, coberturas, calendario de pagos, saldo pendiente, siniestros asociados, fotos de daños, etc. Richards y Ford (2020, cap. 7) explican que, antes de DDD, los desarrolladores buscaban la reutilización holística de las entidades comunes, pero que crear

artefactos compartidos genera acoplamiento, dificulta la coordinación y aumenta la complejidad. Una clase así cambiaría por razones de todos los departamentos y no sería coherente con ninguno.

4.2. División en contextos delimitados

Evans (2003) define el *Bounded Context* como el límite explícito dentro del cual un modelo es válido y tiene un significado preciso. Richards y Ford (2020, cap. 7) lo resumen así: todo lo relacionado con el dominio es visible internamente, pero opaco para los demás contextos; en lugar de crear una clase unificada para toda la organización, cada dominio crea la suya y concilia las diferencias en los puntos de integración.

Para el caso de seguros, el equipo propone los siguientes contextos, cada uno con su propio **Lenguaje Ubicuo**:

Contexto	**Qué significa "póliza"**	**Elementos del modelo (DDD)**
Suscripción y Pólizas	Contrato legal con cláusulas, coberturas, vigencia y prima calculada	Agregado `Poliza` (raíz) con entidades `Cobertura` y `Clausula`; objetos-valor `Vigencia`, `Vehiculo`, `Prima`
Facturación de primas	Cuenta que genera ingresos periódicos	Agregado `CuentaDePrima` con `PlanDePagos`, `Recibo`; objetos-valor `Dinero`, `NumeroPoliza`
Siniestros	Respaldo de cobertura para un reclamo	Agregado `Siniestro` con `Reclamacion`, `Reserva`; objeto-valor `CoberturaVigente` (copia de lo necesario)
Peritaje en campo	Un número de referencia	Agregado `InformeDePeritaje` con `Fotografia`, `DanoEvaluado`; objeto-valor `NumeroPoliza`

Cuadro 2: Contextos delimitados propuestos para el sistema de seguros. Elaboración propia.

La identificación de estos contextos se apoya en dos criterios que proponen Richards y Ford (2020, cap. 17) para encontrar fronteras: el *propósito* (cada contexto aporta un comportamiento significativo y cohesivo) y las *transacciones* (las entidades que deben cooperar en una misma transacción indican una buena frontera). Por ejemplo, emitir una póliza y calcular su prima es una transacción coherente dentro de Suscripción; registrar un pago es una transacción de Facturación que no necesita bloquear la póliza.

La Guía de Arquitectura N-Capas plantea la misma idea con el concepto de *módulos*: cuando el modelo de dominio crece, se particiona en módulos verticales, cada uno responsable de un área de negocio, como ocurre en los ERP con Nómina, Facturación o Almacén. El nombre de cada módulo debe formar parte del Lenguaje Ubicuo, y la comunicación entre módulos debe hacerse mediante interfaces, por ejemplo un Servicio DDD, en lugar de llamar directamente a sus objetos internos (De la Torre et al., 2010, pp. 71–72).

4.3. Cómo evitar que el modelo de Póliza se contamine

Para que el modelo de `Poliza` del contexto de Suscripción no se llene de detalles de siniestros o facturación, el equipo propone las siguientes reglas:

1. **Cada contexto tiene su propio modelo y su propio almacenamiento.** En un contexto delimitado, las partes internas (código y esquemas de datos) están acopladas entre sí, pero nunca a nada externo, como una base de datos o una definición de clase de otro contexto (Richards & Ford, 2020, cap. 17). Siniestros no lee la tabla de pólizas; mantiene su propia copia de lo que necesita.

2. **Preferir la duplicación al acoplamiento.** Richards y Ford (2020, cap. 17) señalan que, cuando el objetivo es un alto desacoplamiento, se favorece la duplicación frente a la reutilización. Que `NumeroPoliza` exista como objeto-valor en tres contextos no es un defecto, sino una decisión deliberada.

3. **Referenciar entre contextos solo por identidad.** Siniestros no guarda una referencia a la entidad `Poliza` de otro contexto, sino un objeto-valor `NumeroPoliza`. Esto es coherente con la regla de los agregados: los objetos externos solo pueden referenciar a la entidad raíz, y solo las raíces tienen repositorios (De la Torre et al., 2010, pp. 172–173).

4. **Usar objetos-valor para lo descriptivo.** La Guía recuerda que un mismo concepto puede ser entidad en un sistema y objeto-valor en otro; su ejemplo es la dirección (De la Torre et al., 2010, pp. 168–169). Aquí ocurre lo mismo con la póliza: es entidad (con identidad y ciclo de vida) en Suscripción, pero solo un valor de referencia en Peritaje.

4.4. Comunicación desacoplada entre contextos

Los contextos necesitan intercambiar información: Facturación debe enterarse de que se emitió una póliza; Siniestros debe saber si una póliza estaba vigente en la fecha del accidente. El equipo propone combinar dos mecanismos:

a) Eventos de dominio (comunicación asíncrona). Cuando una póliza se emite, se cancela o se renueva, el contexto de Suscripción publica un evento (`PolizaEmitida`, `PolizaCancelada`). Facturación y Siniestros se suscriben y actualizan su propio modelo. La Guía define un evento como “un cambio significativo de estado” y describe sistemas con emisores y consumidores de eventos débilmente acoplados (De la Torre et al., 2010, pp. 43–44); también propone los Eventos del Dominio para articular reglas del tipo “cuando [algo] ocurre, el sistema debe [hacer algo]” (p. 195). Richards y Ford (2020, cap. 7) añaden que la comunicación asíncrona, mediante una cola de mensajes, permite absorber las diferencias de capacidad entre servicios.

b) Consultas a través de una interfaz publicada (comunicación síncrona). Cuando el ajustador está en el lugar del accidente y necesita confirmar la vigencia, la aplicación de

Peritaje consulta una API de solo lectura que expone Suscripción. Esa API devuelve un objeto de transferencia simple, nunca la entidad interna.

En ambos casos, el contexto receptor **traduce** el mensaje externo a su propio modelo. Evans (2003) denomina *capa anticorrupción* a esta capa de traducción, que impide que los conceptos de otro modelo se filtren en el propio.

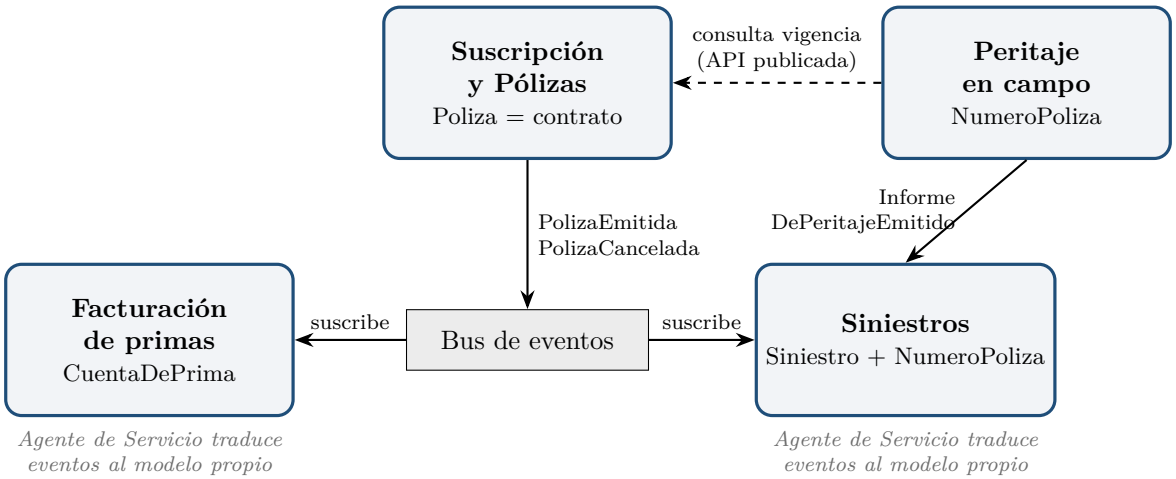


Figura 3: Mapa de contextos propuesto. Cada contexto mantiene su propio modelo de "póliza"; la integración se hace mediante eventos y APIs publicadas, con traducción en el receptor. Elaboración propia.

4.5. Relación con una arquitectura orientada al dominio y el Agente de Servicio

Cada contexto delimitado se implementa internamente con una arquitectura orientada al dominio, como Onion o Clean. Esto es coherente con lo que plantean Richards y Ford (2020, cap. 8): en una partición por dominios, cada componente de primer nivel (aquí, cada contexto) puede tener subcomponentes, incluidas capas, pero la partición superior gira alrededor del dominio. Martin (2017, p. 246) llega a la misma conclusión desde otro ángulo: los servicios deben diseñarse con una arquitectura interna de componentes que respete la Regla de Dependencia.

Dentro del contexto de Siniestros, por ejemplo, la estructura quedaría así:

- **Dominio (centro):** agregado `Siniestro`, objeto-valor `CoberturaVigente`, regla "no se abre un siniestro sobre una póliza cancelada". Aquí se declara también el puerto `IConsultaCoberturas`, expresado en el lenguaje de Siniestros.
- **Aplicación:** caso de uso `AbrirSiniestro`, que usa el puerto sin saber cómo se implementa.
- **Infraestructura (exterior):** la clase `AgenteServicioPolizas`, que implementa `IConsultaCoberturas` llamando a la API de Suscripción, y el consumidor de eventos que actualiza la copia local de coberturas.

Esta clase es lo que la Guía de Arquitectura N-Capas denomina **Agente de Servicio**: objetos que manejan la semántica específica de la comunicación con servicios externos, aíslan a

la aplicación de las particularidades de cada servicio y realizan mapeos entre el formato de datos que expone el servicio externo y el formato que utiliza la aplicación; además pueden incorporar caché o soporte para escenarios sin conexión (De la Torre et al., 2010, pp. 108, 155–156). La Guía añade que, en aplicaciones grandes, los agentes actúan como un nivel de abstracción entre la Capa de Dominio y los servicios remotos, y que son componentes "perfectos" para desacoplarlos con IoC y simularlos con *fakes* en las pruebas (p. 156).

Visto desde la Arquitectura Hexagonal, `IConsultaCoberturas` es un puerto secundario y `AgenteServicioPolizas` es su adaptador. Visto desde DDD, el agente es la capa anticorrupción: convierte el "contrato con cláusulas" de Suscripción en la "cobertura vigente" que Siniestros necesita. Las tres visiones describen el mismo mecanismo.

```
// --- Contexto Siniestros: capa de Dominio ---
public record NumeroPoliza(string Valor);
public record CoberturaVigente(NumeroPoliza Poliza, bool CubreDanosPropios,
                               decimal Franquicia, DateOnly Desde, DateOnly
                                   Hasta);

public interface IConsultaCoberturas   // puerto en lenguaje de Siniestros
{
    Task<CoberturaVigente?> ObtenerAsync(NumeroPoliza poliza, DateOnly fecha);
}

// --- Contexto Siniestros: capa de Infraestructura ---
public class AgenteServicioPolizas : IConsultaCoberturas
{
    private readonly HttpClient _http;
    public AgenteServicioPolizas(HttpClient http) => _http = http;

    public async Task<CoberturaVigente?> ObtenerAsync(NumeroPoliza poliza,
        DateOnly fecha)
    {
        // DTO propio del contexto Suscripcion: nunca sale de esta clase
        var dto = await _http.GetFromJsonAsync<PolizaDto>(
            $"/api/polizas/{poliza.Valor}?fecha={fecha:yyyy-MM-dd}");
        if (dto is null || dto.Estado != "VIGENTE") return null;

        // Traduccion (anticorrupcion): contrato con clausulas -> cobertura
            vigente
        bool danosPropios = dto.Clausulas.Any(c => c.Codigo == "TODO_RIESGO");
        return new CoberturaVigente(poliza, danosPropios, dto.Franquicia,
                                    dto.InicioVigencia, dto.FinVigencia);
    }
}
```

Listado 1: Puerto definido en el dominio de Siniestros y Agente de Servicio en infraestructura (simplificado).

Gracias a esta separación, si el departamento de Suscripción reestructura sus cláusulas o cambia su API, solo cambia `AgenteServicioPolizas`. El agregado `Siniestro` y el caso de uso `AbrirSiniestro` no se enteran.

5. Pregunta 4. La inversión de dependencias en la práctica

5.1. Situación actual

En el sistema de pedidos, la clase `OrderProcessor` contiene la lógica principal de negocio y, para completar su tarea, crea y llama directamente a `AddressValidatorService`, que a su vez depende de una librería HTTP concreta. El Listado 2 muestra una versión simplificada.

```
public class OrderProcessor
{
    public void Process(Order order)
    {
        var validator = new AddressValidatorService();   // dependencia concreta
        if (!validator.Validate(order.ShippingAddress))
            throw new InvalidOperationException("Direccion invalida");

        order.MarkAsConfirmed();
        // ... resto de la logica de negocio
    }
}

public class AddressValidatorService
{
    private readonly SomeHttpLibrary.Client _client = new("https://api.validador
        .com");

    public bool Validate(Address address)
    {
        var response = _client.Post("/validate", address);
        return response.StatusCode == 200 && response.Body.Contains("\"valid\":
            true");
    }
}
```

Listado 2: Diseño original: dependencia directa de una clase concreta.

5.2. a) Por qué este diseño viola los principios de Hexagonal, Clean y Onion

Dirección de las dependencias. En las tres arquitecturas, las dependencias de código fuente deben apuntar **hacia el núcleo de negocio**. Martin (2017, p. 203) lo establece como Regla de Dependencia; Palermo (2008c) lo enuncia como "la dirección del acoplamiento es hacia el centro"; Cockburn (2005) lo expresa como que el código del interior no debe depender de la tecnología del exterior. En el diseño actual ocurre lo contrario: `OrderProcessor` (política de alto nivel) depende de `AddressValidatorService` (detalle de bajo nivel), que a su vez depende de una librería HTTP. Por transitividad, la regla de negocio está acoplada al protocolo HTTP, a la URL del proveedor y al formato JSON de su respuesta.

Violación del DIP. Martin (2017, p. 87) define el Principio de Inversión de Dependencias diciendo que los sistemas más flexibles son aquellos en los que las dependencias de código fuente se refieren solo a abstracciones, no a concreciones. Entre sus reglas prácticas figuran: no referirse a clases concretas volátiles y *no mencionar nunca el nombre de nada concreto y volátil* (Martin, 2017, p. 89). La línea `new AddressValidatorService()` viola ambas: menciona y crea una clase concreta. Además, un servicio externo de validación es exactamente el tipo de elemento volátil que Martin distingue de los elementos estables como `String`, que sí se toleran (p. 88).

La Guía de Arquitectura N-Capas expresa el mismo principio en sus dos enunciados: las capas de alto nivel no deben depender de las de bajo nivel, ambas deben depender de abstracciones; y las abstracciones no deben depender de los detalles, sino los detalles de las abstracciones (De la Torre et al., 2010, p. 64). Insiste además en que, entre capas, no se deben instanciar dependencias de forma explícita (p. 66).

Consecuencias concretas.

- No es posible probar `OrderProcessor` sin conexión a Internet ni sin que el servicio externo esté disponible.
- Un cambio en la librería HTTP o en la API del proveedor obliga a recompilar y redesplegar la lógica de pedidos.
- No se puede cambiar de proveedor sin modificar código de negocio, lo que viola también el Principio Abierto/Cerrado (Martin, 2017, p. 70).

5.3. b) Rediseño

La solución consiste en introducir una abstracción que exprese *lo que el negocio necesita*, no *cómo se obtiene*. El negocio necesita saber si una dirección es entregable; no le interesa si eso se resuelve con HTTP, con una base de datos local o con una regla fija.

```
// ===== Proyecto Pedidos.Core (no referencia a ninguna libreria HTTP) =====
public record AddressValidationResult(bool IsValid, string? Reason = null);

public interface IAddressValidator          // PUERTO: definido por el negocio
{
    Task<AddressValidationResult> ValidateAsync(Address address);
}

public class OrderProcessor
{
    private readonly IAddressValidator _addressValidator;

    // La dependencia llega desde fuera (inyeccion por constructor)
    public OrderProcessor(IAddressValidator addressValidator)
        => _addressValidator = addressValidator;

    public async Task ProcessAsync(Order order)
    {
```

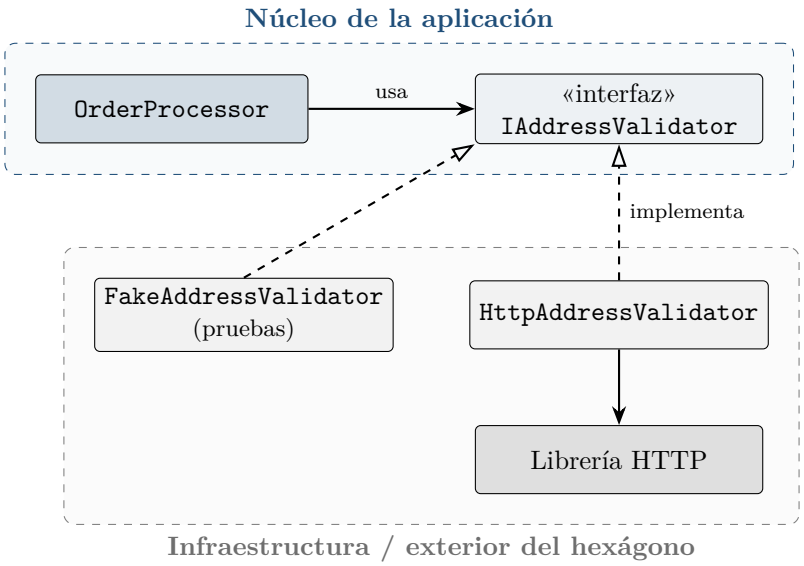


Figura 4: Rediseño: la interfaz pertenece al núcleo; la implementación HTTP y el doble de prueba pertenecen al exterior. Todas las flechas que cruzan la frontera apuntan hacia el núcleo. Elaboración propia.

```
        var result = await _addressValidator.ValidateAsync(order.ShippingAddress
            );
        if (!result.IsValid)
            throw new InvalidAddressException(result.Reason);

        order.MarkAsConfirmed();
    }
}
```

Listado 3: Núcleo de la aplicación: entidad, puerto y lógica de negocio.

```
// ===== Proyecto Pedidos.Infrastructure (referencia a Pedidos.Core) =====
public class HttpAddressValidator : IAddressValidator   // ADAPTADOR
{
    private readonly HttpClient _http;
    public HttpAddressValidator(HttpClient http) => _http = http;

    public async Task<AddressValidationResult> ValidateAsync(Address address)
    {
        var response = await _http.PostAsJsonAsync("/validate", address);
        if (!response.IsSuccessStatusCode)
            return new(false, "Servicio de validacion no disponible");

        var body = await response.Content.ReadFromJsonAsync<ProviderResponse>();
        return new(body!.Valid, body.Message);   // traduccion al modelo del
            nucleo
    }
}

// ===== Program.cs (componente "main": unico lugar que conoce las concreciones)
    =====
builder.Services.AddHttpClient<IAddressValidator, HttpAddressValidator>(
```

```
    c => c.BaseAddress = new Uri("https://api.validador.com"));
builder.Services.AddScoped<OrderProcessor>();
```

Listado 4: Infraestructura: adaptador concreto y composición en el punto de arranque.

Justificación de la ubicación de cada componente.

- `IAddressValidator` reside en el **núcleo** porque es un contrato que define el negocio según sus propias necesidades. La Guía indica que los contratos que definen el comportamiento requerido a los componentes de bajo nivel *deben existir en los ensamblados de alto nivel*; cuando los componentes de bajo nivel los implementan, son ellos los que dependen, al compilar, de los de alto nivel; por eso se habla de inversión (De la Torre et al., 2010, pp. 64–65).
- `HttpAddressValidator` reside en la **infraestructura**, o fuera del hexágono, porque está ligado a una tecnología concreta. Es un adaptador secundario en términos de Cockburn (2005) y pertenece a la capa de adaptadores de interfaz en términos de Martin (2017, p. 205), donde se convierten los datos de agentes externos al formato interno.
- El **componente de composición** (`Program.cs`) es el único que conoce las clases concretas. Martin (2017, p. 91) reconoce que las violaciones del DIP no pueden eliminarse por completo, pero sí concentrarse en un número reducido de componentes concretos, normalmente llamado `main`.

Un detalle de diseño importante es que el puerto devuelve `AddressValidationResult`, un tipo del núcleo, y no la respuesta HTTP ni el DTO del proveedor. De lo contrario, el núcleo volvería a conocer formatos del exterior, lo que Martin (2017, p. 207) desaconseja expresamente.

5.4. c) Pruebas aisladas, mantenibilidad y cambio de proveedor

Con el nuevo diseño, `OrderProcessor` se puede probar inyectando un doble de prueba que implemente `IAddressValidator`. No se realizan llamadas HTTP reales, no se depende de la librería HTTP y las pruebas se ejecutan en milisegundos.

```
public class FakeAddressValidator : IAddressValidator
{
    private readonly bool _valid;
    public int Calls { get; private set; }
    public FakeAddressValidator(bool valid) => _valid = valid;

    public Task<AddressValidationResult> ValidateAsync(Address address)
    {
        Calls++;
        return Task.FromResult(new AddressValidationResult(_valid, _valid ? null
            : "No existe"));
    }
}

public class OrderProcessorTests
```

```
{
    [Fact]
    public async Task Confirma_el_pedido_cuando_la_direccion_es_valida()
    {
        var processor = new OrderProcessor(new FakeAddressValidator(valid: true)
            );
        var order = OrderMother.WithAddress("Calle 23 No. 456");

        await processor.ProcessAsync(order);

        Assert.True(order.IsConfirmed);
    }

    [Fact]
    public async Task Rechaza_el_pedido_cuando_la_direccion_es_invalida()
    {
        var processor = new OrderProcessor(new FakeAddressValidator(valid: false
            ));
        var order = OrderMother.WithAddress("Direccion inexistente");

        await Assert.ThrowsAsync<InvalidAddressException>(() => processor.
            ProcessAsync(order));
        Assert.False(order.IsConfirmed);
    }
}
```

Listado 5: Prueba unitaria de OrderProcessor con un doble de prueba (xUnit).

Esto es exactamente lo que plantea Cockburn (2005) cuando afirma que el beneficio último de los puertos y adaptadores es poder ejecutar la aplicación en modo completamente aislado. La Guía añade que, mediante la inyección de dependencias, este cambio de implementación real a simulada puede hacerse incluso por configuración (De la Torre et al., 2010, p. 65).

Implicaciones para la mantenibilidad.

- **Cambios aislados:** actualizar la librería HTTP, cambiar la URL o el formato de respuesta del proveedor afecta solo a `HttpAddressValidator`.
- **Responsabilidad única:** `OrderProcessor` contiene solo reglas de pedidos; la comunicación de red queda en otra clase que cambia por otros motivos.
- **Comportamientos transversales sin tocar el negocio:** se puede añadir caché, reintentos o un *circuit breaker* envolviendo el adaptador (por ejemplo, un `CachedAddressValidator` que también implemente `IAddressValidator`), sin modificar `OrderProcessor`.

Cambio de proveedor. Si mañana se cambia de proveedor, se escribe un nuevo adaptador (por ejemplo, `OtroProveedorAddressValidator`) y se cambia una línea en `Program.cs`. El núcleo no se modifica ni se recompila. Esto cumple el Principio Abierto/Cerrado: el comportamiento del sistema se extiende sin modificar el artefacto existente (Martin, 2017, p. 70).

Incluso es posible tener dos proveedores a la vez, uno por país, inyectando la implementación adecuada según la configuración, que es uno de los escenarios que la Guía cita como ejemplo de desacoplamiento dentro del propio dominio (De la Torre et al., 2010, p. 64).

Como contrapartida, el equipo reconoce que el diseño añade una interfaz, una clase y un registro en el contenedor. Para una aplicación pequeña puede ser excesivo. Pero para una dependencia de red externa, volátil y fuera de nuestro control, el coste es mínimo frente al beneficio.

6. Pregunta 5. El *Kitty Problem* y el Principio Abierto/Cerrado

6.1. Aclaración sobre la fuente

La orientación del seminario atribuye el *Kitty Problem* a Richards y Ford. Sin embargo, al revisar la bibliografía, el equipo comprobó que este ejemplo aparece en *Clean Architecture*, en el capítulo 27, "Services: Great and Small" (Martin, 2017, pp. 242–246). En *Fundamentals of Software Architecture* no se encontró ninguna referencia a este caso. Por esa razón, en esta sección se cita a Martin como fuente original del ejemplo y a Richards y Ford como apoyo en los conceptos de partición por dominio y *bounded context*, que sí desarrollan ampliamente.

6.2. Descripción del problema

Martin (2017, pp. 242–243) describe un sistema agregador de taxis que conoce a varios proveedores de una ciudad y permite a los clientes pedir viajes según criterios como hora de recogida, coste, lujo o experiencia del conductor. Para que el sistema fuera escalable, los arquitectos lo construyeron con muchos microservicios pequeños, cada uno a cargo de un equipo:

- `TaxiUI`: atiende a los clientes desde sus dispositivos móviles.
- `TaxiFinder`: examina los inventarios de los `TaxiSupplier` y determina qué taxis son candidatos.
- `TaxiSelector`: aplica los criterios del usuario y elige un taxi entre los candidatos.
- `TaxiDispatcher`: ordena el taxi seleccionado.

Tras un año de funcionamiento, el departamento de marketing anuncia un servicio de entrega de gatitos a domicilio. Se establecerán puntos de recogida de gatitos; cuando se haga un pedido, un taxi cercano recogerá un gatito y lo entregará. Solo algunos proveedores participan; los conductores alérgicos a los gatos no deben ser seleccionados; y un vehículo que haya transportado gatitos en los últimos tres días no debe asignarse a clientes que declaren alergia.

La pregunta de Martin es directa: ¿cuántos servicios deben cambiar? Su respuesta: *todos*. El desarrollo y despliegue de la nueva funcionalidad tendrá que coordinarse cuidadosamente, lo que demuestra que los servicios *están acoplados* y no pueden desarrollarse, desplegarse ni mantenerse de forma independiente (Martin, 2017, p. 243).

6.3. Por qué la división por verbos o funciones llevó al problema

Martin (2017, p. 243) identifica la causa: es el problema de las *preocupaciones transversales* (*cross-cutting concerns*). Las descomposiciones funcionales como la del sistema de taxis son muy vulnerables a nuevas funcionalidades que atraviesan todos esos comportamientos.

El equipo considera útil explicarlo así: los servicios se dividieron según *pasos de un proceso* (encontrar, seleccionar, despachar), es decir, según verbos. Cada uno de esos pasos contiene implícitamente el conocimiento de *qué se transporta*: personas. Cuando aparece un nuevo tipo de carga (gatitos), cambia la semántica de cada paso: encontrar ahora implica filtrar proveedores participantes y conductores alérgicos; seleccionar implica considerar el historial del vehículo; despachar implica pasar por un punto de recogida. La variación del negocio ("qué se transporta") atraviesa perpendicularmente la descomposición elegida ("qué paso del proceso es").

Este fenómeno guarda relación con lo que Richards y Ford (2020, cap. 8) observan en las arquitecturas particionadas técnicamente: un flujo de negocio como *CatalogCheckout* queda "esparcido" entre todas las capas. Aquí ocurre lo mismo, pero con servicios: el concepto de negocio "entrega de gatitos" queda repartido entre todos los servicios. Además, también se acerca a lo que estos autores llaman la *trampa de la entidad*: diseñar componentes como una simple correspondencia con las entidades o funciones técnicas, en lugar de con los flujos de negocio (Richards & Ford, 2020, cap. 8).

Martin (2017, p. 241) nombra este error como la *falacia del desacoplamiento*: creer que, por estar separados físicamente en servicios, los componentes están desacoplados. Si están acoplados por datos o por comportamiento, su desarrollo, despliegue y operación deben coordinarse igualmente.

6.4. Solución: componentes `Rides` y `Kittens` como *plugins*

Martin (2017, p. 244) propone resolver el problema con una arquitectura basada en componentes guiada por los principios SOLID: un conjunto de clases que puedan extenderse polimórficamente para manejar nuevas funcionalidades. La estrategia es:

1. Conservar en clases base abstractas la lógica genérica de los servicios originales: encontrar candidatos, seleccionar y despachar un vehículo.
2. Extraer la lógica específica de los viajes de personas a un componente `Rides`.
3. Colocar la nueva funcionalidad en un componente `Kittens`.
4. Ambos componentes sobrescriben las clases base abstractas mediante patrones como *Template Method* o *Strategy*, y respetan la Regla de Dependencia: dependen de las abstracciones, nunca al revés.
5. Las clases concretas son creadas por fábricas bajo control de la interfaz de usuario.

El resultado, según Martin (2017, p. 244), es que al implementar la funcionalidad de gatitos solo cambia `TaxiUI`; nada más necesita modificarse. Se añade un nuevo `jar`, *gem* o DLL al

sistema y se carga dinámicamente. La funcionalidad queda desacoplada y puede desarrollarse y desplegarse de forma independiente.

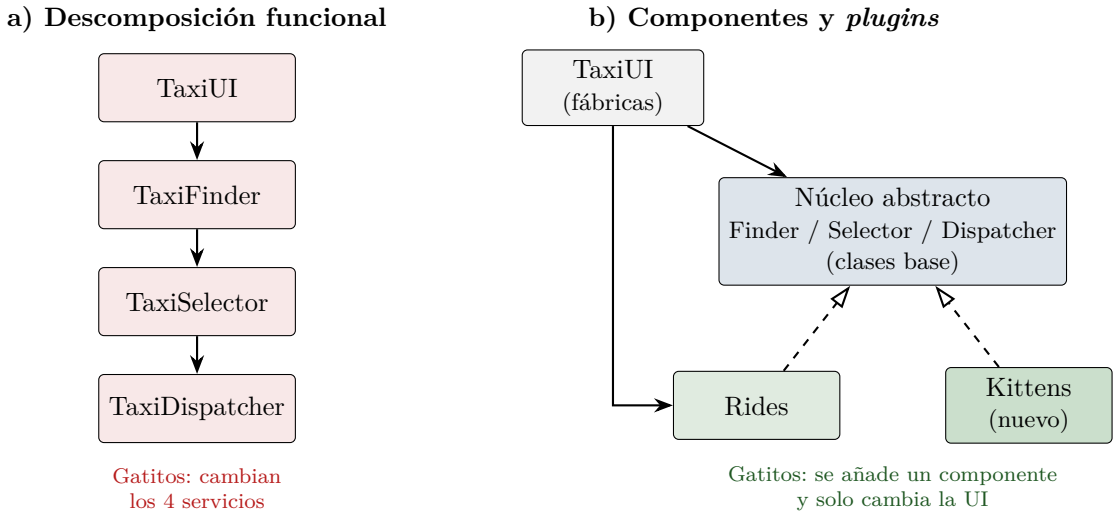


Figura 5: Kitty Problem. En (a) la nueva funcionalidad atraviesa todos los servicios. En (b) se añade como un componente que extiende abstracciones del núcleo. Elaboración propia a partir de Martin (2017, pp. 242–246).

```
// ===== Componente nucleo (estable, cerrado a modificacion) =====
public abstract class TransportRequest { public Location Pickup { get; init; } }

public interface ICandidateFilter      { bool Accepts(Vehicle v,
    TransportRequest r); }
public interface IDispatchPlan        { IEnumerable<Stop> BuildRoute(
    TransportRequest r); }

public abstract class TransportService      // Template Method
{
    protected abstract IEnumerable<ICandidateFilter> Filters { get; }
    protected abstract IDispatchPlan Plan { get; }

    public Vehicle Handle(TransportRequest request, IEnumerable<Vehicle> fleet)
    {
        var candidate = fleet.First(v => Filters.All(f => f.Accepts(v, request))
            );
        candidate.AssignRoute(Plan.BuildRoute(request));
        return candidate;
    }
}

// ===== Componente Rides (comportamiento original extraido) =====
public class RideService : TransportService { /* filtros de lujo, coste, etc. */
    }

// ===== Componente Kittens (nuevo; se despliega como una DLL adicional) =====
public class KittenDeliveryService : TransportService
{
    protected override IEnumerable<ICandidateFilter> Filters => new
```

```
        ICandidateFilter[] {
27          new ParticipatingSupplierFilter(), new DriverNotAllergicFilter() };
28      protected override IDispatchPlan Plan => new ViaCollectionPointPlan();
29  }
30  // La regla "vehiculo con gatitos en los ultimos 3 dias" se agrega como un
        filtro
31  // adicional en Rides para clientes alergicos, sin modificar el nucleo.
```

Listado 6: Esbozo del núcleo abstracto y de los componentes que lo extienden.

6.5. Relación con el OCP y con DDD / Onion / Clean

Principio Abierto/Cerrado. Martin (2017, p. 70) define el OCP como: *un artefacto de software debe estar abierto a la extensión pero cerrado a la modificación.* Y advierte que, si extensiones simples de los requisitos obligan a cambios masivos en el software, los arquitectos han fracasado de manera espectacular. El *Kitty Problem* es justamente ese fracaso. Con la solución propuesta, los servicios pueden pensarse como un conjunto de clases abstractas en uno o varios `jar`, y cada nueva funcionalidad como otro `jar` con clases que las extienden. Desplegar una funcionalidad deja de ser redesplegar servicios y pasa a ser añadir ficheros a la ruta de carga: *añadir funcionalidades se ajusta al Principio Abierto/Cerrado* (Martin, 2017, p. 246).

Papel de los adaptadores en la extensión. En la solución, `Rides` y `Kittens` cumplen el mismo papel que un adaptador en la Arquitectura Hexagonal: se conectan a una abstracción definida por el núcleo (el "puerto" que forman las clases base y las interfaces como `ICandidateFilter`) y aportan una implementación concreta. El núcleo no conoce sus nombres, y por tanto cumplen la Regla de Dependencia. La selección del componente concreto se hace en el punto de composición: en la propuesta de Martin, las fábricas controladas por la UI. Es la misma idea de arquitectura de *plugins* que Martin (2017, pp. 170–171) presenta para la base de datos y la interfaz gráfica: las reglas de negocio centrales se mantienen separadas e independientes de los componentes opcionales o que pueden implementarse de muchas formas.

Aporte de DDD. Desde DDD, el error inicial fue trazar las fronteras según la mecánica del proceso y no según el lenguaje del negocio. Si se hubiera dialogado con los expertos del dominio, habría aparecido un concepto más general que "viaje": por ejemplo, *servicio de transporte*, del cual el viaje de pasajeros es una variante. Modelar explícitamente ese concepto en el núcleo, con sus variaciones como extensiones, es lo que permite absorber el cambio.

La lección arquitectónica. Martin (2017, pp. 246–247) concluye que las fronteras arquitectónicas *no caen entre servicios, sino que los atraviesan*, dividiéndolos en componentes. Los servicios, por útiles que sean para la escalabilidad, no son por sí mismos elementos arquitectónicamente significativos: la arquitectura se define por las fronteras internas y por las dependencias que las cruzan. Para el equipo, esta es probablemente la idea más importante de toda la pregunta: *separar físicamente no es lo mismo que desacoplar.*

7. Pregunta 6. N-Capas frente a requisitos de atributos de calidad

7.1. El punto de partida

Richards (2015, pp. 8–9) califica la arquitectura en capas con una puntuación **baja** en agilidad general, facilidad de despliegue, rendimiento y escalabilidad, y **alta** en testabilidad y facilidad de desarrollo. En la obra posterior, Richards y Ford (2020, cap. 10) mantienen la valoración de una estrella en elasticidad y escalabilidad, y dos en rendimiento, y además rebajan la testabilidad, porque en un monolito grande pocos desarrolladores ejecutan toda la batería de regresión para un cambio pequeño.

Un estudiante propone mitigar estas debilidades con **capas abiertas** y la técnica *fast-lane reader*. El equipo analiza la propuesta a continuación.

7.2. Qué son las capas abiertas y el *fast-lane reader*

En una arquitectura en capas, cada capa puede ser *cerrada* o *abierta*. Una capa cerrada obliga a que la petición pase por ella antes de llegar a la siguiente; una capa abierta permite saltarla (Richards, 2015, pp. 3–5). Richards y Ford (2020, cap. 10) explican que, para consultas simples, sería más rápido que la presentación accediera directamente a la base de datos, saltándose las capas intermedias; a principios de los 2000 esto se conocía como el patrón *fast-lane reader*. Para lograrlo, las capas de negocio y persistencia tendrían que ser abiertas.

7.3. Efecto sobre el Aislamiento de Capas y la mantenibilidad

El concepto de **capas de aislamiento** (*layers of isolation*) significa que los cambios en una capa no afectan a componentes de otras capas, siempre que los contratos entre ellas se mantengan (Richards & Ford, 2020, cap. 10). Pero los autores son claros: *para garantizar el aislamiento, las capas que participan en el flujo principal de la petición tienen que ser cerradas.* Si la presentación accede directamente a la persistencia, los cambios en la persistencia afectan tanto al negocio como a la presentación, lo que da lugar a una aplicación muy acoplada, frágil y costosa de cambiar (Richards, 2015, pp. 3–4).

Por tanto, abrir capas para ganar rendimiento implica **sacrificar directamente el aislamiento**, que es la principal virtud del estilo. A largo plazo:

- Un cambio en el esquema de la base de datos ya no queda contenido en la capa de persistencia, sino que se propaga hacia la interfaz de usuario. Este es precisamente el síntoma del sistema de inventario de la Pregunta 1.
- La lógica de negocio comienza a duplicarse: si la presentación lee directamente, tarde o temprano aparecerán filtros o cálculos de negocio en la presentación.
- Las reglas de acceso se vuelven difíciles de gobernar. Richards (2015, p. 5) advierte que no

documentar o comunicar qué capas son abiertas y cerradas, y por qué, suele dar lugar a arquitecturas acopladas y frágiles, muy difíciles de probar, mantener y desplegar.

Hay que precisar que Richards (2015, p. 4) sí admite capas abiertas en casos concretos, como una capa de servicios compartidos que el negocio puede saltarse para llegar a la persistencia. La diferencia está en abrir *una capa de apoyo* de forma documentada, frente a abrir *el flujo principal* para evitar trabajo.

7.4. El *sinkhole anti-pattern* y la regla 80/20

El antipatrón **sumidero de arquitectura** (*architecture sinkhole*) describe el caso en que las peticiones atraviesan varias capas como simple paso de datos, sin que ninguna aplique lógica. Por ejemplo, la presentación pide datos del cliente, el negocio los pide a la persistencia, la persistencia ejecuta un SQL y los datos suben sin transformación alguna (Richards, 2015, p. 7). Esto provoca instanciación y procesamiento innecesarios que afectan al consumo de memoria y al rendimiento (Richards & Ford, 2020, cap. 10).

La **regla 80/20** sirve para diagnosticarlo: toda arquitectura en capas tendrá algunos casos de sumidero, y es aceptable que alrededor del 20 % de las peticiones sean de simple paso. Si la proporción se invierte y el 80 % de las peticiones son sumideros, es una señal de que la arquitectura en capas no es el estilo adecuado para el dominio del problema (Richards & Ford, 2020, cap. 10).

El análisis del equipo es que la propuesta del estudiante *confunde el síntoma con la causa*. Las capas abiertas son, según los propios autores, una forma de mitigar el sumidero, pero con el coste explícito de que *será más difícil controlar el cambio por la falta de aislamiento* (Richards, 2015, pp. 7–8). Si el sistema tiene mayoritariamente sumideros, abrir las capas no corrige el diagnóstico: confirma que el estilo no se ajusta al problema. Y si el sistema tiene sobre todo peticiones con lógica, abrir capas no aporta casi nada, porque esas peticiones deben pasar por el negocio de todos modos. En ningún caso se resuelve la raíz.

7.5. Por qué las capas abiertas no resuelven la escalabilidad ni la agilidad

Más allá del rendimiento local, el problema de fondo es estructural:

- **Escalabilidad.** La arquitectura en capas es siempre un único *quantum* arquitectónico, con interfaz, lógica y base de datos monolíticas, de modo que solo escala hasta cierto punto (Richards & Ford, 2020, cap. 10). Se puede replicar la aplicación completa en varios nodos, pero la granularidad es demasiado gruesa y resulta cara (Richards, 2015, p. 9). Saltarse capas no cambia el número de unidades desplegables ni permite escalar una parte del sistema sin escalar el resto.
- **Facilidad de despliegue.** Un cambio de tres líneas exige redesplegar toda la unidad, con el riesgo de que otros cambios se cuelen en el mismo despliegue (Richards & Ford, 2020, cap. 10). Abrir capas no modifica esta situación.

- **Agilidad.** Es baja por la naturaleza monolítica y el acoplamiento fuerte (Richards, 2015, p. 8). Las capas abiertas *aumentan* el acoplamiento, así que empeoran la agilidad.

En resumen, el *fast-lane reader* es una optimización local de rendimiento que se paga con mantenibilidad, y que no afecta a los atributos que realmente penalizan al estilo: escalabilidad, despliegue y agilidad.

7.6. Por qué Hexagonal o Clean son más adecuadas, y con qué matices

Para un sistema con requisitos de alta agilidad y escalabilidad, el equipo considera que las arquitecturas Hexagonal y Limpia, combinadas con DDD, son más adecuadas por las siguientes razones:

1. **Agilidad por dependencias invertidas.** Como la infraestructura depende del núcleo y no al revés, los cambios tecnológicos quedan localizados en adaptadores (Martin, 2017, p. 203; Palermo, 2008c). Las reglas de negocio se prueban en aislamiento y con rapidez (Martin, 2017, p. 202; Cockburn, 2005), lo que reduce el riesgo de cada cambio y permite cambiar más a menudo.
2. **Partición por dominio.** Al combinarse con DDD, el sistema se organiza en contextos delimitados y no en capas técnicas. Richards y Ford (2020, cap. 8) señalan que la partición por dominio se modela de forma más cercana al funcionamiento del negocio, se alinea con los estilos de monolito modular y microservicios, y *facilita migrar datos y componentes a una arquitectura distribuida.*
3. **Escalabilidad como evolución posible.** Un contexto delimitado bien aislado, con su propio modelo, sus propios datos y comunicación por puertos, puede extraerse como servicio independiente y escalarse por separado. Richards y Ford (2020, cap. 17) describen los microservicios como la encarnación física de los conceptos lógicos de DDD, donde cada servicio modela un *bounded context.* Un monolito en capas técnicas no ofrece esas costuras: el dominio está esparcido por todas las capas.

No obstante, el equipo considera necesario matizar la afirmación de que estas arquitecturas sean *inherentemente* más escalables. Hexagonal, Clean y Onion son, ante todo, formas de organizar las *dependencias de código fuente*, no topologías de despliegue. Una aplicación Clean desplegada como un único ejecutable con una única base de datos sigue siendo un solo *quantum* y escala igual que un monolito en capas. Martin (2017, p. 247) insiste en que la arquitectura se define por las fronteras y las dependencias, no por los mecanismos físicos de despliegue. Y Palermo (2008a) aclara que su enfoque no es apropiado para sitios pequeños.

Por tanto, la conclusión más precisa es la siguiente: estas arquitecturas **no garantizan** la escalabilidad, pero sí la **hacen posible** con un coste razonable, porque crean las fronteras por las que el sistema puede dividirse cuando haga falta. En cambio, sí mejoran de forma directa la agilidad y la testabilidad. La arquitectura en capas con optimizaciones, en cambio, sigue estando limitada por su naturaleza monolítica y técnicamente particionada.

Atributo	**N-Capas (capas cerradas)**	**N-Capas + capas abiertas / fast-lane**	**Hexagonal / Clean + DDD**
Rendimiento en lecturas simples	Bajo (sumidero)	Mejora local	Similar; se puede añadir un adaptador de lectura sin romper la Regla de Dependencia
Aislamiento / mantenibilidad	Medio	Empeora	Alto
Agilidad	Baja	Empeora	Alta
Facilidad de despliegue	Baja	Sin cambios	Depende del despliegue; los contextos pueden separarse
Escalabilidad	Baja (un solo *quantum*)	Sin cambios	Habilitada por la partición en contextos
Testabilidad del negocio	Media	Empeora	Alta

Cuadro 3: Valoración comparativa del equipo, basada en Richards (2015, pp. 8–9), Richards y Ford (2020, caps. 8, 10 y 17) y Martin (2017, pp. 202–207).

8. Conclusiones

El análisis de las seis preguntas permite al equipo extraer varias conclusiones generales:

1. **La diferencia esencial entre los estilos estudiados no está en la forma del dibujo, sino en la dirección de las dependencias.** N-Capas pone la base de datos en el fondo y hace que todo dependa de ella. Hexagonal, Onion y Clean ponen el dominio en el centro y hacen que la infraestructura dependa de él. Palermo (2008c) lo muestra al dibujar las capas como círculos y las cebollas como capas: la forma es intercambiable, la estructura no.

2. **Hexagonal, Onion y Clean son variaciones de una misma idea.** Martin (2017, p. 202) reconoce que estas arquitecturas tienen el mismo objetivo, la separación de responsabilidades, y producen sistemas independientes de *frameworks*, de la interfaz, de la base de datos y de cualquier agente externo, además de testables. Cada una aporta un énfasis distinto: Cockburn, la simetría interior-exterior; Palermo, el modelo de dominio en el centro; Martin, la formulación precisa de la regla.

3. **El Principio de Inversión de Dependencias es el mecanismo que las hace posibles.** Las preguntas 1, 2 y 4 muestran el mismo patrón: definir la interfaz en el núcleo, implementarla en el exterior y componer en un único punto.

4. **DDD aporta el criterio para trazar las fronteras.** Las arquitecturas anteriores dicen *cómo* proteger el núcleo; DDD dice *qué* es el núcleo y dónde termina cada modelo. Las preguntas 3 y 5 muestran que unas fronteras mal elegidas, ya sea un modelo único de póliza o servicios divididos por verbos, generan acoplamiento aunque los componentes estén separados físicamente.

5. **Ningún estilo es superior en todos los casos.** La arquitectura N-Capas sigue siendo una opción razonable para aplicaciones pequeñas, centradas en datos o con poca lógica de negocio (Richards & Ford, 2020, cap. 10; De la Torre et al., 2010, p. 42). Las arquitecturas orientadas al dominio añaden complejidad que solo se justifica cuando la lógica del negocio es compleja y el sistema debe vivir y evolucionar durante años.

En definitiva, elegir una arquitectura es elegir qué se quiere que sea fácil de cambiar. Si lo que más cambia es la tecnología, la infraestructura debe estar en el exterior; si lo que más cambia es el negocio, el negocio debe estar bien delimitado en contextos. Las arquitecturas estudiadas, combinadas con DDD, permiten conseguir ambas cosas a la vez, a cambio de una mayor disciplina de diseño.

Referencias bibliográficas

Cockburn, A. (2005). *Hexagonal architecture (Ports and adapters)* [HaT Technical Report 2005.02]. Recuperado de `https://alistair.cockburn.us/hexagonal-architecture/`

De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. A., & Calvarro Nelson, J. (2010). *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0* (Borrador). Microsoft Ibérica; Krasis Consulting.

Evans, E. (2003). *Domain-Driven Design: Tackling complexity in the heart of software*. Addison-Wesley.

Martin, R. C. (2017). *Clean Architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Palermo, J. (2008a, 29 de julio). *The Onion Architecture: part 1*. Programming with Palermo. `https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/`

Palermo, J. (2008b, 30 de julio). *The Onion Architecture: part 2*. Programming with Palermo. `https://jeffreypalermo.com/2008/07/the-onion-architecture-part-2/`

Palermo, J. (2008c, 4 de agosto). *The Onion Architecture: part 3*. Programming with Palermo. `https://jeffreypalermo.com/2008/08/the-onion-architecture-part-3/`

Richards, M. (2015). *Software architecture patterns*. O'Reilly Media.

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Declaración de Uso de la Inteligencia Artificial

En la elaboración del presente documento se utilizaron las herramientas de inteligencia artificial **Gemini** y **ChatGPT**. Su propósito principal fue realizar resúmenes y facilitar una mejor comprensión de la información aportada por la bibliografía recomendada, dado que las fuentes consultadas son considerablemente extensas.

El contenido de este trabajo fue extraído por los integrantes del equipo mediante el estudio de las fuentes bibliográficas recomendadas y redactado por ellos. Posteriormente, se empleó la inteligencia artificial para enriquecer y mejorar la redacción de los textos.

La totalidad del contenido fue revisada, analizada y verificada en conjunto por los integrantes del equipo, quienes asumen la responsabilidad final del documento. A pesar del uso de la inteligencia artificial como herramienta de apoyo, los integrantes tienen pleno dominio de los temas tratados en cada una de las preguntas.